

# Module 1 — Supermarket & Receipt Processing

---

## 1. Purpose and Scope

Ingest supermarket invoices (PDF, photo or manual entry), identify and normalize every product, categorize it into the 3-tier pt-PT taxonomy, track consumption and price evolution, and prevent duplicates and flag shrinkflation — so the household knows what it actually spends on groceries without typing it in.

This is the **crown-jewel ingestion module**. The receipt review screen is the flow the entire automation-first thesis is judged on: data arrives messy, the system parses, matches and scores it, **auto-accepts what it is confident about**, and routes only the uncertain remainder to the shared Review Queue.

**There is no accuracy target.** Every line and every receipt carries a visible confidence, and a status screen shows exactly what is waiting for a human. The auto-accept rate is *reported* so it can be watched improving — it is never a gate to pass. See Decision #41.

Unlike M9, this module cannot be flat. It carries six tables because each answers a question a field cannot: what the invoice claimed (**Receipt**) versus what the items sum to (**ReceiptItem**), what a product *is* (**MasterProduct**) versus what a merchant *calls* it (**ProductAlias**), how a given merchant's layout is parsed (**MerchantParserProfile**), and an immutable price observation (**ProductPriceHistory**) that must survive a later correction to its source receipt. Everything beyond that is deliberately a field, not a table.

**Processing is local by default.** Parsing, OCR, normalization, matching and scoring each have a local engine that works with no subscription and no network, and those are what ship enabled. A remote engine is a supported *option*, not a forbidden one: every stage sits behind a provider seam and a remote implementation can be registered and switched on per stage once a subscription exists. Nothing degrades while it stays off — see Decision #13.

All monetary fields are EUR only. Scope is supermarket and grocery retail; other receipt types must not influence the initial domain model.

**Dependencies.** Every shared entity this module touches — **Entity**, **Merchant**, **Category**, **Tag**, **Document**, **Link**, **ReviewTask**, **Setting**, **AuditLog**, **ImportBatch**, **ProcessingJob** — is defined once in the orchestrator brief §1a and is **consumed, never redefined here**. If one does not exist yet when this module is built, it is created by whichever phase owns it, to the §1a definition. The full list is in the Integration Contract at the end of this file. This module assumes nothing about build order.

---

## Build This in Three Passes

**This module is too large for one build pass and must not be attempted as one.** Six tables, nineteen functional requirements, ten screens, a Celery pipeline, four merchant parsers, OCR, a classifier and a confidence engine — for comparison, all of M9 was three tables and four screens.

The **document is not split**; the *work* is. Each pass reads this whole file plus the orchestrator brief and is told which slice to build. Every slice ends demoable, migrates cleanly, and leaves **./fm check** green.

| Slice                                          | Builds                                                                                         | Tables                                                 | FRs                                                               | Screens                     | Exit criterion                                                                                                                                                                        |
|------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>M1a —<br/>Ingest &amp;<br/>Review</b>       | The spine:<br>upload →<br>parse →<br>score →<br>review →<br>confirm                            | <b>Receipt, ReceiptItem,<br/>MerchantParserProfile</b> | 1.1,<br>1.2,<br>1.6,<br>1.7,<br>1.8,<br>1.15,<br>1.16             | UX-1.1,<br>1.2, 1.3         | A Continente PDF and a<br>Lidl PDF upload, parse,<br>reconcile to their printed<br>totals, and confirm.<br>Lines are raw text +<br>prices; no products yet                            |
| <b>M1b —<br/>Products &amp;<br/>Categories</b> | The brain:<br>resolution,<br>learning,<br>taxonomy,<br>and the<br>import<br>that fills<br>them | <b>MasterProduct,<br/>ProductAlias</b>                 | 1.3,<br>1.9,<br>1.11,<br>1.12,<br>1.13,<br>1.17,<br>1.18,<br>1.19 | UX-1.4,<br>1.5, 1.6,<br>1.7 | The 2025 import yields<br>~1,564 products; a re-<br>parse of the fixtures<br>now resolves products<br>and categories, and a<br>correction sticks                                      |
| <b>M1c —<br/>Money<br/>over time</b>           | The<br>payoff:<br>price<br>trends, Fs,<br>loyalty,<br>ledger<br>links                          | <b>ProductPriceHistory</b>                             | 1.4,<br>1.5,<br>1.10,<br>1.14<br>+<br>the<br>Fs<br>rules          | UX-1.8,<br>1.9,<br>1.10     | €/kg trends render<br>across merchants,<br>shrinkflation fires on a<br>seeded case, and an Fs<br>article changes<br><b>notional_total_eur</b><br>without touching<br><b>total_eur</b> |

**Why this order.** M1a is the crown jewel and the only slice worth demoing alone. M1b must follow it because product resolution needs real parsed lines to resolve. M1c must follow M1b because a price history keyed by product cannot exist before products do. The legacy import sits in M1b, not last, because it is the only thing that turns an empty catalogue into a useful one (Decision #39).

## Invariants — break these and the data is silently wrong

Everything else in this file is detail. These nine are not.

1. **Money is **NUMERIC(10,2)** decimal EUR.** Never float, never minor units. Display via the shared pt-PT util only.
2. **paid\_price\_eur is always what was actually paid** — **0.00** on an Fs article, negative on a refund. It is never a notional or list value.
3. **Adding an Fs article changes no printed figure.** **total\_eur, computed\_total\_eur, is\_reconciled** and the **item\_count** check must be byte-identical before and after.
4. **Only pvp – promo – invoice\_allocated reconciles.** Summing per-item promos and forgetting the invoice-level credit is the classic bug in this domain.
5. **A category lives on MasterProduct and nowhere else** (Decision #34). A line inherits it by resolving; an unresolved line has none, and that is the truth.

6. **description\_norm** is a matching key and is never displayed. **description\_raw** is verbatim audit trail. The user sees **canonical\_name**.
7. **Duplicates are prevented, not detected**: **UNIQUE (entity\_id, atcud\_code)** plus SHA-256 idempotency (Decision #35).
8. **ProductPriceHistory** is **append-only**. A correction writes a new row; the past is never rewritten.
9. **Every stage has a local engine that works offline and unsubscribed** (Decision #13). A remote engine is optional and per-stage.

---

## Actors

### Primary Actor

A member of the household who buys groceries and wants that spending tracked accurately without typing it in. The same person photographs the receipt, resolves whatever the system flagged, corrects a mismatched product, and later looks at what a kilo of coffee now costs. There is **one human actor wearing several hats at different moments** — shopper, reviewer, price analyst, catalogue maintainer — not several actors: they carry identical permissions, share one workspace, and no workflow branches on which hat is being worn.

Read/write power is governed solely by the M7 role (**OWNER/MEMBER** write, **VIEWER** reads); that is a cross-cutting concern of the household module and is not re-specified here. Dependents (children) never act — they are only ever the *subject* of an attributed record.

### Supporting Actor

This module is automation-first, so one non-human actor does most of the work:

- **The ingestion pipeline** — selects a parser, extracts, normalizes, matches, classifies, scores confidence and proposes reconciliation links, unattended, inside a Celery **ProcessingJob**. It is the actor in most user stories below; the human confirms rather than types. Every stage runs on a local engine unless a remote one has been explicitly configured.

---

## User Stories

Traceability only — the behaviour is specified in the Functional Requirements, and each row names the FRs that satisfy it.

| #    | As a user I want...                                                                                                                      | FRs         |
|------|------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| US01 | to photograph or drop in a receipt and have merchant, date, lines, weights, prices and discounts extracted, so I never retype an invoice | 1.1,<br>1.6 |
| US02 | to see only the fields the system was unsure about, side by side with the original, so I confirm a receipt in seconds                    | 1.1,<br>1.2 |
| US03 | to ask "porquê?" of any filled-in value and see the reasons and scores, so I can trust it or correct it informed                         | 1.1,<br>1.2 |

| #    | As a user I want...                                                                                                                            | FRs          |
|------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| US04 | a product I re-matched by hand remembered for that merchant, and autocomplete over what I already own so I never create a near-duplicate       | 1.8          |
| US05 | every item placed in my own pt-PT category tree, auto-suggested and always overridable                                                         | 1.3          |
| US06 | to add the articles that were <i>not</i> on the invoice, tag them Fs and record their worth — then filter any view to Fs, non-Fs, or all       | 1.2,<br>1.5  |
| US07 | to see how a product's price per kilo moved across merchants and seasons, in list price and in what I paid                                     | 1.4,<br>1.12 |
| US08 | to be told when a product I buy regularly got lighter while the price held or rose                                                             | 1.4          |
| US09 | a receipt I already uploaded recognised and <b>refused outright</b> , so a double upload cannot inflate my spending                            | 1.9          |
| US10 | the cartão discount tracked and spread correctly across the items it applied to, apart from per-item promos                                    | 1.10         |
| US11 | to filter what I bought by dietary attributes and allergens                                                                                    | 1.5          |
| US12 | a receipt matched to the statement line that paid for it — automatically when obvious, by picker when not                                      | 1.14         |
| US13 | my years of <b>SUPERMARKET_YYYY</b> rows imported without duplication and re-runnable if it goes wrong                                         | 1.19         |
| US14 | to drop several invoices at once, walk away, and come back to see which parsed, which need me and which failed — retrying without re-uploading | 1.16         |
| US15 | Continente receipts read by a Continente parser and Lidl's by its own, falling back to a generic one                                           | 1.15         |
| US16 | the whole pipeline to work on my own NAS with no paid service, and to plug a better engine in later without rebuilding                         | 1.1,<br>1.15 |
| US17 | to browse one row per invoice <b>or</b> one row per article across every invoice, jumping from any article to the original image               | 1.18         |
| US18 | to see which categories the system guessed and which I confirmed, so I can work through the rest                                               | 1.3          |
| US19 | to rename, move, merge and retire categories as my thinking changes, and understand what that does to existing data                            | 1.17         |

## Data Model

### Entities & Attributes

All monetary attributes are EUR only (no multi-currency, no FX). Stored as **NUMERIC(10,2)** decimal EUR (e.g. **19.99**); the UI displays them pt-PT-formatted (e.g. **19,99 €**) through the shared money util only.

Every row carries `entity_id` — the owning household entity. Entity is an **attribution and filter** dimension, not a security boundary: every household member can read every entity's receipts (see M7).

**Definition — the Fs split.** Everything the invoice prints is a normal paid item. An **Fs article was never on the invoice**: the household adds it by hand afterwards, attached to that receipt. It carries a **notional value** — what it would have cost — and `ReceiptItem.is_fs` marks it.

**Only document parsing is skipped, not automation.** An Fs article is entered through the same product autocomplete as any other line, so once it resolves to a `MasterProduct` it inherits that product's category, weights and last known price automatically. What it cannot have is anything the *document* would have supplied: `line_no`, `merchant_section`, `iva_class_raw`.

| Rule                                                | Consequence                                                                                                                                                                                                      |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>paid_price_eur = 0.00</code> — no money moved | Adding one cannot disturb <code>total_eur</code> , <code>computed_total_eur</code> or a reconciliation that already passed                                                                                       |
| The value lives in <code>unit_price_pvp_eur</code>  | <code>notional_value_eur = is_fs ? pvp × quantity : paid_price_eur</code> — the measure every "what was it worth" view uses, Fs or not                                                                           |
| It is not a printed line                            | The printed <code>item_count</code> is checked against non-Fs rows only and never moves; <code>fs_item_count</code> counts Fs separately                                                                         |
| It is a real product consumed                       | Quantity, cost and price history include it, subject to the <code>fs</code> filter ( <code>only</code>   <code>exclude</code>   <code>all</code> , default <code>all</code> ) that every query and screen offers |

Worked example — 10 paid articles totalling €10, plus 2 Fs articles worth €5 each: `total_eur = 10.00`, `fs_value_eur = 10.00`, `notional_total_eur = 20.00`, `fs_share_pct = 100 %`.

The notional value is pre-filled from the product's last observed price and is otherwise typed in; `notional_value_source` records which. **The figure is taken at face value** — it enters `ProductPriceHistory` as a normal observation and counts towards €/kg trends and shrinkflation, because what makes this safe is not exclusion but visibility: `is_fs` travels onto the snapshot so any analysis can run with Fs, without, or over both.

⚠ On an Fs snapshot **both** `list_price_eur` and `paid_price_eur` carry the notional value, never `0.00`. The receipt item records what was *spent*; the price history records what the product was *worth*. Writing the literal zero would drag every paid-price trend for that product towards zero.

⚠ **Fs is an overloaded token on real receipts — never pattern-match it.** Portuguese *faturas simplificadas* are numbered `FS 013700526/091423`, and **Lidl uses F as an IVA class letter meaning 0 %** (`Deposito 0.10 0,10 F`). Neither has anything to do with the household's Fs flag, which is never printed and never parsed.

**Definition — `is_fs` is neither a `product_flag` nor a tag.** Every `product_flag` value is read off the document and an Fs article has no document origin at all; `tags[]` are open user labels (`#férias`, `#culinária`) with no effect on any arithmetic. The household's own taxonomy file draws the same line, calling its context flags "*distintas da flag de exclusão Fs*".

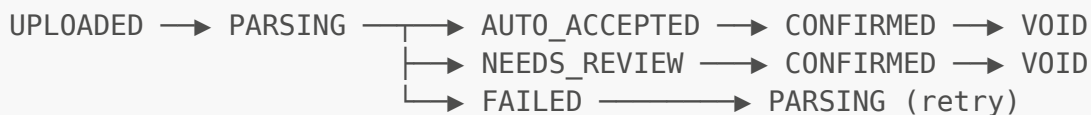
**Definition — claimed versus computed.** `Receipt.total_eur`, `total_discount_eur` and `item_count` are **what the invoice printed** — independent facts extracted from the document. The item rows are what we parsed. Reconciliation is the comparison of the two, and it is a *confidence signal*, never a silent correction of either side. Everything Fs-related is **computed from the rows**, never claimed, because nothing about it is printed.

## 1. **Receipt** (One Payment Transaction at One Merchant)

| Attribute                       | Type          | Description                                                                          |
|---------------------------------|---------------|--------------------------------------------------------------------------------------|
| <code>id</code>                 | UUID          | Primary key                                                                          |
| <code>entity_id</code>          | UUID          | Owning entity (attribution/filter)                                                   |
| <code>merchant_id</code>        | UUID          | FK → Core <b>Merchant</b> ; must already exist                                       |
| <code>purchased_at</code>       | Timestamp     | Instant of purchase, stored UTC                                                      |
| <code>purchase_date</code>      | Date          | Local <b>Europe/Lisbon</b> calendar date — <b>never UTC-shifted</b>                  |
| <code>document_id</code>        | UUID          | FK → Core <b>Document</b> (the original PDF/image)                                   |
| <code>parser_profile_id</code>  | UUID?         | FK → <b>MerchantParserProfile</b> actually used; <b>NULL</b> when parsed generically |
| <code>import_batch_id</code>    | UUID?         | FK → Core <b>ImportBatch</b> when bulk-imported                                      |
| <code>processing_job_id</code>  | UUID?         | FK → Core <b>ProcessingJob</b> for the parse run                                     |
| <code>total_eur</code>          | NUMERIC(10,2) | Printed invoice total actually paid                                                  |
| <code>total_discount_eur</code> | NUMERIC(10,2) | Printed total discount; drives the proration ratio                                   |
| <code>item_count</code>         | Integer       | Printed article count; cross-checked against non-Fs rows only                        |
| <code>status</code>             | Enum          | See the lifecycle below                                                              |
| <code>confidence</code>         | NUMERIC(4,3)? | Combined score, 0–1                                                                  |
| <code>decision_reasons</code>   | JSONB         | <code>[{rule, detail, score}]</code> — rendered behind "Porquê?"                     |
| <code>raw_ocr_payload</code>    | JSONB?        | Provider output; <b>never selected in list queries</b>                               |
| <code>atcud_code</code>         | String?       | Portuguese tax stamp read from the QR code                                           |
| <code>atcud_valid</code>        | Boolean?      | Structural validation result                                                         |
| <code>atcud_reason</code>       | String?       | Why validation failed, when it did                                                   |

| Attribute                                          | Type          | Description                                                                             |
|----------------------------------------------------|---------------|-----------------------------------------------------------------------------------------|
| <code>parsed_payment_methods</code>                | JSONB         | <code>[{method, amount_eur}]</code> for composite payments                              |
| <code>loyalty_scheme</code>                        | String?       | e.g. <code>"Cartão Continente"</code> ; <code>NULL</code> when the receipt carried none |
| <code>loyalty_card_masked</code>                   | String?       | Exactly as printed ( <code>"XXXXXXXX3394X"</code> ) — never a full number               |
| <code>loyalty_accrued_eur</code>                   | NUMERIC(10,2) | Credited to the card by this purchase ( <code>default: 0.00</code> )                    |
| <code>loyalty_discount_eur</code>                  | NUMERIC(10,2) | Card balance spent on this receipt ( <code>default: 0.00</code> )                       |
| <code>tags</code>                                  | UUID[]        | Core <code>Tag</code> ids                                                               |
| <code>notes</code>                                 | Text?         | Free-text notes                                                                         |
| <code>void_reason</code>                           | Text?         | Mandatory when <code>status = VOID</code>                                               |
| <code>is_deleted</code>                            | Boolean       | Soft-delete flag ( <code>default: false</code> )                                        |
| <code>created_at / updated_at / deleted_at?</code> | Timestamp     | Audit timestamps                                                                        |

**Status lifecycle** — a guarded state machine; illegal transitions are rejected and every change writes an `AuditLog` row:



### Constraints & Rules:

- `AUTO_ACCEPTED` is reached with no human input when `confidence ≥ confidence.auto_accept`. The threshold is a `Setting`, so the household decides how much nagging it wants.
- `NEEDS_REVIEW` creates exactly one `ReviewTask` (`module = "receipts"`, `subject_type = "Receipt"`).
- `VOID` replaces deletion once a receipt is confirmed; `void_reason` is required.
- Re-uploading a file whose SHA-256 already exists for this entity never creates a second receipt.
- Unique on (`entity_id`, `atcud_code`) when `atcud_code IS NOT NULL`** — the ATCUD is the fiscal document identity, so this is what makes duplicates impossible rather than merely detectable (Decision #35).
- Unique on (`entity_id`, `document_id`).

## 2. ReceiptItem (One Purchased Article)

| Attribute                       | Type            | Description                                                                                                                               |
|---------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                 | UUID            | Primary key                                                                                                                               |
| <code>receipt_id</code>         | UUID            | FK → <b>Receipt</b>                                                                                                                       |
| <code>entity_id</code>          | UUID            | Denormalized from the parent (see Decision #8)                                                                                            |
| <code>line_no</code>            | Integer?        | Position on the invoice; <b>NULL for an Fs item</b> , which was never printed                                                             |
| <code>merchant_section</code>   | String?         | The merchant's own heading above this line (" <b>Mercearia Salgada</b> ", " <b>FRUTAS E VEGETAIS</b> ") — a no-cost classification signal |
| <code>description_raw</code>    | String          | Text exactly as printed; for an Fs item, what the user typed                                                                              |
| <code>description_norm</code>   | String          | Accent/case/unit-stripped form used for matching — <b>the only key a receipt line gives us</b>                                            |
| <code>master_product_id</code>  | UUID?           | FK → <b>MasterProduct</b> ; <b>NULL</b> until resolved. <b>The category comes with it</b> (Decision #34)                                  |
| <code>quantity</code>           | NUMERIC (14,4)  | As printed; negative for refunds                                                                                                          |
| <code>unit</code>               | Enum            | <b>KG</b>   <b>G</b>   <b>L</b>   <b>ML</b>   <b>UN</b>   <b>PACK</b>                                                                     |
| <code>quantity_canonical</code> | NUMERIC (14,4)  | Normalized quantity                                                                                                                       |
| <code>unit_canonical</code>     | Enum            | <b>KG</b>   <b>L</b>   <b>UN</b> — families never convert between each other                                                              |
| <code>weight_listed_kg</code>   | NUMERIC (14,4)? | Label weight, proposed from the matching <b>MasterProduct.pack_variants</b> entry                                                         |
| <code>weight_observed_kg</code> | NUMERIC (14,4)? | Real weight from the register scale                                                                                                       |
| <code>is_bulk_weighed</code>    | Boolean         | Priced at the checkout scale ( <b>default: false</b> )                                                                                    |
| <code>unit_price_pvp_eur</code> | NUMERIC (10,2)  | List/gross price before any discount; on an Fs item, its <b>notional value</b>                                                            |
| <code>promo_discount_eur</code> | NUMERIC (10,2)  | Per-item promotion only ( <b>default: 0.00</b> )                                                                                          |
| <code>promo_type</code>         | Enum?           | <b>ABSOLUTE</b>   <b>PERCENTAGE</b>   <b>BOGO</b>                                                                                         |

| Attribute                      | Type          | Description                                                                                     |
|--------------------------------|---------------|-------------------------------------------------------------------------------------------------|
| invoice_allocated_discount_eur | NUMERIC(10,2) | Invoice/loyalty discount <b>prorated</b> onto this item                                         |
| paid_price_eur                 | NUMERIC(10,2) | Net spend after all discounts; <b>always 0.00 on an Fs item</b>                                 |
| iva_class_raw                  | String(4)?    | The IVA token <b>exactly as printed</b> (A, C, 6%) — captured, never interpreted (Decision #38) |
| is_fs                          | Boolean       | <b>Fs item</b> — added by hand, never on the invoice ( <b>default: false</b> )                  |
| notional_value_source          | Enum?         | PRICE_HISTORY   MANUAL — where an Fs item's value came from                                     |
| product_flag                   | Enum?         | REFUND   DEPOSIT_RETURN   SEASONAL   OTHER — all read off the document                          |
| tags                           | UUID[]        | Core Tag ids                                                                                    |
| notes                          | Text?         | Free-text notes (legacy Notas)                                                                  |
| legacy_row_ref                 | Integer?      | Spreadsheet row id, traceability only — <b>not</b> the PK                                       |
| confidence                     | NUMERIC(4,3)? | Per-item score                                                                                  |
| decision_reasons               | JSONB         | Per-item reasons                                                                                |
| is_deleted                     | Boolean       | Soft-delete flag ( <b>default: false</b> )                                                      |
| created_at / updated_at        | Timestamp     | Audit timestamps                                                                                |

### Price arithmetic — three distinct quantities that must never be collapsed:

| Field                          | Formula                                | Legacy column |
|--------------------------------|----------------------------------------|---------------|
| unit_price_pvp_eur             | source                                 | Price         |
| promo_discount_eur             | source                                 | PromoInd      |
| invoice_allocated_discount_eur | (pvp - promo) × invoice_discount_ratio | PromoGlob     |
| paid_price_eur                 | pvp - promo - invoice_allocated        | Price_Final   |

### Constraints & Rules:

- The most common bug in this domain is summing per-item promos and missing the invoice-level credit. Only **pvp - promo - invoice\_allocated** reconciles.

- **Three names, three jobs.** `description_raw` is verbatim and never modified — it is the audit trail against the paper. `description_norm` is a matching key and is **never displayed**. The name shown to the user is `MasterProduct.canonical_name` once resolved, falling back to `description_raw` while unresolved.
- `category_status` lives on the product: `AUTO` until a human confirms the suggestion (`VALIDATED`) or assigns directly (`MANUAL`). `MANUAL` counts as validated for every filter and KPI.
- `iva_eur` is **not modelled at all** — see Decision #38.
- Refunds carry `quantity < 0` and are excluded from the paid total; the refund count is derived, not stored.
- Deposits (*tara*) use `product_flag = DEPOSIT_RETURN` and are never categorized as groceries.

### 3. MasterProduct (Canonical Product Identity, User-Managed)

| Attribute                             | Type                        | Description                                                                                                                                     |
|---------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                       | <code>UUID</code>           | Primary key                                                                                                                                     |
| <code>canonical_name</code>           | <code>String</code>         | Household's name for the product; <b>the name shown everywhere</b>                                                                              |
| <code>brand</code>                    | <code>String?</code>        | Brand; lives here, never in the category tree                                                                                                   |
| <code>category_l3_id</code>           | <code>UUID</code>           | <b>Deepest category — authoritative, and the only place a category is stored</b>                                                                |
| <code>category_l1_id / _l2_id</code>  | <code>UUID</code>           | Maintained ancestors, recomputed on reparent                                                                                                    |
| <code>category_status</code>          | <code>Enum</code>           | <code>AUTO</code>   <code>VALIDATED</code>   <code>MANUAL</code> ( <code>default: AUTO</code> ) — confirmed once per product, not once per line |
| <code>category_confidence</code>      | <code>NUMERIC(4,3)?</code>  | Score behind an <code>AUTO</code> assignment                                                                                                    |
| <code>pack_variants</code>            | <code>JSONB</code>          | <code>[{label?, weight_kg?, barcode?}]</code> — every pack size this product ships in                                                           |
| <code>sold_by_weight</code>           | <code>Boolean</code>        | Priced per kg at the counter (bananas, fish, legumes) rather than per pack ( <code>default: false</code> )                                      |
| <code>dietary_attributes</code>       | <code>JSONB</code>          | <code>ORGANIC</code>   <code>VEGAN</code>   <code>GLUTEN_FREE</code>   ...                                                                      |
| <code>allergen_list</code>            | <code>JSONB</code>          | Allergens                                                                                                                                       |
| <code>seasonal_flags</code>           | <code>JSONB</code>          | <code>SPRING</code>   <code>SUMMER</code>   <code>AUTUMN</code>   <code>WINTER</code>                                                           |
| <code>expected_shelf_life_days</code> | <code>Integer?</code>       | Stored now; alerting deferred                                                                                                                   |
| <code>deposit_value_eur</code>        | <code>NUMERIC(10,2)?</code> | Bottle/packaging deposit                                                                                                                        |
| <code>is_deleted</code>               | <code>Boolean</code>        | Soft-delete flag ( <code>default: false</code> )                                                                                                |
| <code>created_at / updated_at</code>  | <code>Timestamp</code>      | Audit timestamps                                                                                                                                |

**Immutable rule: two items with different categories are, by rule, different master products.** L3 is the normalized product-*genus*; brand, variant and package live here. That rule is precisely why the category lives on the product and nowhere else — see Decision #34.

**Last known price (derived, never stored).** To make manual entry — and Fs valuation — fast, every product exposes its most recent observation from `ProductPriceHistory`: `last_pvp_eur`, `last_weight_kg`, `last_observed_on`, and `last_price_per_kg_eur`. The UI pre-fills whichever fits: **the pack price for packaged goods, the €/kg for anything sold by weight**, because a banana or a *courgette* has no meaningful pack price. Both figures are printed on real receipts — Piquete prints **Preco: 1,99/KG**, Lidl prints **1,19 EUR/kg** — so this is captured, not guessed. It stays derived because it is the newest row of a table we already keep, and storing it would create a second truth to drift.

**One product, many pack sizes (Decision #15).** A 500 g and a 1 kg bag of the same coffee are **one MasterProduct** — that is what makes €/kg comparable across them. What differs between them is the **weight**, and `pack_variants` is what lets one product carry several:

```
[{"label": "500 g", "weight_kg": 0.500}, {"label": "1 kg", "weight_kg": 1.000}]
```

Receipts embed the size in the description itself (**POLPA TOMATE GULOSO 500G**, **SAL GROSSO CONTINENTE 1KG**), so the variant is matched by parsing that text against these weights — there is no code to join on. `barcode` remains an accepted optional key on a variant for the deferred scan-to-find flow, but **nothing in the ingestion pipeline reads it** — see Decision #23. This single field replaces what were three (`barcode`, `sku_reference`, `curated_weights`) and one that never earned its keep (`curated_isoweight_equivalents`).

4. ProductAlias (Learned Merchant Vocabulary)

| Attribute               | Type         | Description                                   |
|-------------------------|--------------|-----------------------------------------------|
| id                      | UUID         | Primary key                                   |
| master_product_id       | UUID         | FK → MasterProduct                            |
| merchant_id             | UUID         | FK → Core Merchant                            |
| merchant_description    | String       | Raw text as that merchant prints it           |
| description_norm        | String       | Normalized form actually matched against      |
| confidence              | NUMERIC(4,3) | Learned trust, rises with corrections         |
| correction_count        | Integer      | How many times a human confirmed this mapping |
| last_used_at            | Timestamp?   | Last successful match                         |
| created_at / updated_at | Timestamp    | Audit timestamps                              |

Unique constraint: (`merchant_id`, `description_norm`). Aliases are a table rather than a JSON blob on the product precisely because they are **learned** and carry per-merchant confidence.

## 5. ProductPriceHistory (Immutable Price Observation)

| Attribute              | Type           | Description                                                                                           |
|------------------------|----------------|-------------------------------------------------------------------------------------------------------|
| id                     | UUID           | Primary key                                                                                           |
| master_product_id      | UUID           | FK → MasterProduct                                                                                    |
| merchant_id            | UUID           | FK → Core Merchant                                                                                    |
| observed_on            | Date           | Purchase date of the source receipt                                                                   |
| is_fs                  | Boolean        | Snapshot of the source item's Fs status, so price analysis can filter without a join (default: false) |
| weight_kg              | NUMERIC(14,4)? | Weight used for the €/kg figures                                                                      |
| list_price_eur         | NUMERIC(10,2)  | Snapshot of the list price; the notional value on an Fs row                                           |
| paid_price_eur         | NUMERIC(10,2)  | Snapshot of what was paid; <b>also the notional value on an Fs row</b> , never 0.00                   |
| source_receipt_item_id | UUID           | Provenance                                                                                            |
| created_at             | Timestamp      | Audit timestamp                                                                                       |

**Append-only and immutable.** Correcting a receipt item later writes a **new** row and never rewrites the past. €/kg is **not** stored here: price and weight sit on the same row, so the division cannot drift and storing it would only add columns to keep in step. `shrinkflation_indicator` is likewise computed over the 12-month window at query time, not frozen onto a row that cannot know its own future.

## 6. MerchantParserProfile (How One Merchant's Layout Is Read)

| Attribute          | Type   | Description                                                                 |
|--------------------|--------|-----------------------------------------------------------------------------|
| id                 | UUID   | Primary key                                                                 |
| merchant_id        | UUID?  | FK → Core Merchant; <b>NULL = the generic fallback profile</b>              |
| name               | String | e.g. "Continente talão térmico"                                             |
| parser_key         | String | Registered parser implementation, e.g. <code>continente_v1</code>           |
| document_kinds     | JSONB  | Which inputs it handles: <code>PDF_DIGITAL</code>   <code>IMAGE_SCAN</code> |
| detection_patterns | JSONB  | Regexes / anchor strings that identify this merchant's layout               |

| Attribute                            | Type                       | Description                                                                                                                                                           |
|--------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>field_hints</code>             | <code>JSONB</code>         | Layout hints — at minimum <code>line_value_is_net</code> , section-heading and savings-line patterns, the column the value sits in, date format and decimal separator |
| <code>priority</code>                | <code>Integer</code>       | Tie-break when several profiles match (higher wins)                                                                                                                   |
| <code>success_rate</code>            | <code>NUMERIC(4,3)?</code> | Rolling share of receipts this profile auto-accepted                                                                                                                  |
| <code>is_active</code>               | <code>Boolean</code>       | Disable without deleting ( <code>default: true</code> )                                                                                                               |
| <code>is_deleted</code>              | <code>Boolean</code>       | Soft-delete flag ( <code>default: false</code> )                                                                                                                      |
| <code>created_at / updated_at</code> | <code>Timestamp</code>     | Audit timestamps                                                                                                                                                      |

### Constraints & Rules:

- Exactly **one** profile with `merchant_id IS NULL` exists — the generic fallback. It must always be able to run, so a brand-new merchant is never a dead end.
- `parser_key` names a parser **registered in code**; `field_hints` tunes it without a deploy. A merchant-specific parser is a class, not a config blob — configuration alone cannot handle Continente's net line values and Pingo Doce's gross ones equally.
- A profile is **never** required to add a merchant. Absent a match, the generic profile runs and the receipt simply arrives with lower confidence.
- `success_rate` is observed, not configured, and is shown in the UI so a failing profile is visible.

### Observed Receipt Anatomy (from real fixtures)

Eleven real *talões* live in [seed/supermarket/invoices/](#) — four Continente, four Pingo Doce, two Lidl and one Piquete da Fruta. **Read them before writing a parser.** All four merchants print a different layout, and the differences are the strongest possible argument for per-merchant profiles (FR-1.16). The Piquete fixture is a **photograph**, not a PDF, so it is the one that exercises OCR, skew and a thumb over the payment block.

**Continente line format** — `(IVA class) DESCRIPTION VALUE`, with savings on the next line and multi-buys wrapped onto a continuation line:

|                                   |      |                                  |
|-----------------------------------|------|----------------------------------|
| Mercearia Salgada:                |      | ← merchant's own section heading |
| (C) POLPA TOMATE GULOSO 500G      | 1,19 |                                  |
| POUPANCA                          | 0,30 | ← saving, annotation only        |
| (C) BAT FR AZEITE CONTINENTE 150G |      |                                  |
| 2 X 1,45                          | 2,90 | ← quantity × unit price, wrapped |

**Pingo Doce line format** — `IVA class DESCRIPTION [qty X unit] VALUE`, savings parenthesised, weighed goods carrying a 3-decimal quantity:

```

FRUTAS E VEGETAIS          ← merchant's own section heading
C COGUMELOS BRANC 300G  1,000 X 0,74  0,74  ← 1 kg × €0,74/kg
C IO PROT COC GA2X150G      1,29
  Poupança Imediata          (0,10)
DEPÓSITO VOLTA
I  ºVALOR DEPÓSITO          2 X 0,10  0,20  ← tara, IVA class I = 0 %

```

**Lidl line format** — the IVA class sits at the **end** of the line, deposits follow their item, and weighed goods wrap with the €/kg printed:

```

AGUA                        0,32 C      ← class letter last, not first
  Deposito 0.10             0,10 F      ← tara, class F = 0 %
BIO IOGURTE NATURAL 0,37 x 2  0,74 B      ← unit price × qty, inline
BANANA                     0,70 B
  kg x 0,590  1,19 EUR/kg      ← 0,590 kg × 1,19 = 0,70

```

**Piquete da Fruta line format** — quantity **first**, IVA as a literal percentage rather than a letter, and every article sold by weight:

```

QTD  UNI DESCRICAO          IVA  VALOR
--FRUTAS--                  ← section heading again
0,302 KG  NECTARINA MEDIA    6%    0,60
  Preco: 1,99/KG              ← €/kg printed outright
0,45  KIL BATATA OLHO DE     6%    0,67  ← "KIL" and "KG" on one receipt

```

| What the receipts prove                                                                                                                                                  | Consequence                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>The same IVA token appears in three shapes and two meanings.</b> Continente <b>A</b> =6 %, Lidl <b>A</b> =23 %; Lidl <b>F</b> =0 %; Piquete prints <b>6%</b> outright | Reason enough not to model it. The token is captured verbatim and interpreted by nobody (Decision #38) |
| <b>Lidl uses F as an IVA class for 0 %</b> , and every merchant numbers documents <b>FS</b> ... ( <i>fatura simplificada</i> )                                           | Two collisions with the household's Fs flag. Never pattern-match on <b>F/FS</b> (Decision #36)         |
| <b>All four merchants group lines under headings</b> — <b>Mercearia Salgada</b> , <b>FRUTAS E VEGETAIS</b> , <b>--TUBERCULOS--</b>                                       | <b>merchant_section</b> is a universal signal, not a Continente quirk (Decision #27)                   |
| <b>€/kg is printed for weighed goods</b> — <b>Preco:</b> <b>1,99/KG</b> , <b>1,19 EUR/kg</b>                                                                             | Captured, never derived from a guessed weight; it is what pre-fills manual entry                       |
| <b>Units are inconsistent inside one receipt</b> — <b>0,302 KG</b> beside <b>0,45 KIL</b>                                                                                | Unit normalization is required at parse time, not analysis time (FR-1.14)                              |

| What the receipts prove                                                                                                                                                                                                                                                                       | Consequence                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>No EAN, no article code, anywhere</b>                                                                                                                                                                                                                                                      | <b>ProductAlias</b> is the load-bearing match mechanism (Decision #23)               |
| <b>Discount semantics are opposite.</b> Continente: <b>SUBTOTAL</b> already net of <b>POUPANCA</b> , only the cartão subtracted ( $40,56 - 4,00 = 36,56$ ). Pingo Doce: <b>TOTAL</b> gross, <b>POUPANÇA</b> subtracted ( $15,70 - 0,50 = 15,20$ ). Lidl and Piquete print no discounts at all | <b>field_hints.line_value_is_net</b> per profile (Decision #28)                      |
| <b>Two NIFs on one receipt</b> — Lidl prints <b>NIF:503340855</b> (its own) and <b>NIF...: 226443361</b> (the buyer's); Continente does the same                                                                                                                                              | First digit decides: <b>5</b> = company, <b>2</b> = person (Decision #29)            |
| <b>The IVA summary table always foots to the total</b> — Lidl $4,44+2,54+0,32+0,20 = 7,50$ ; Piquete $4,63+0,28 = 4,91$                                                                                                                                                                       | A spare arithmetic anchor, available if line-sum reconciliation ever proves too weak |
| <b>Size lives inside the description</b> — <b>500G, 1KG, 397G, 45GR</b>                                                                                                                                                                                                                       | Weight extraction is description parsing                                             |
| <b>Loyalty appears on one merchant of four</b> — Continente only; Lidl and Piquete print none                                                                                                                                                                                                 | Loyalty is a few nullable fields on <b>Receipt</b> , not a table (Decision #37)      |
| <b>Card numbers are pre-masked</b> — <b>XXXXXXXXX3394X, ****7104/53</b>                                                                                                                                                                                                                       | Confirms Decision #4: nothing sensitive ever reaches us                              |
| <b>ATCUD is printed as text on all eleven</b> — <b>ATCUD: J6K7G7ZS-091423</b>                                                                                                                                                                                                                 | It is the fiscal document identity, and therefore the duplicate key (Decision #35)   |
| <b>No F marker for a household freebie appears anywhere</b>                                                                                                                                                                                                                                   | Expected: Fs articles are not on the invoice at all (Decision #31)                   |

## Extraction Strategy — read this before writing a parser

The layouts above were transcribed with **pdftotext -layout** and, for the photograph, a vision model. **Neither is what will run in production, and the transcriptions in this file must not be treated as a benchmark.** Three rules follow.

**Parse positions, not flat text.** The merchant differences are *positional* — Continente puts the IVA token first, Lidl puts it last, Piquete puts quantity first — so a parser built on string-splitting **-layout** output breaks the moment the extractor's spacing changes. Use **pdfplumber** word boxes (**x0, x1, top**), cluster words into lines by **top**, then assign fields by x-range. That survives an extractor upgrade; column-counted whitespace does not.

**Expect photographs to be far worse than digital PDFs.** **pytesseract** on a curved thermal *talão*, skewed, with a thumb across the payment block, will read **0,302** as **0.302** and lose diacritics — routinely. The ten PDF fixtures and the one photograph are **not the same problem** and must not be judged alike: report the auto-accept rate for digital PDFs and photographs **separately**, and expect image receipts to land in review far more often.

**Pin the extraction, not the parse.** Commit the extracted word boxes for every fixture as golden files. A parser change then shows up as a diff in parsed output alone, and an extractor upgrade as a diff in the golden files — so the two failure modes can never masquerade as each other.

## Category Taxonomy (pt-PT)

- **Language policy.** Schema columns and `code_en` identifiers are **English**; `display_name_pt` carries the household's Portuguese vocabulary verbatim. UI is pt-PT-first and i18n-ready.
- **Level meanings.** **L1** = macro group (**Aperitivos**), **L2** = subcategoria (**Batatas Fritas**), **L3** = **normalized product-genus** (**Batata Frita Ondulada**) — brand-, size- and SKU-agnostic.
- **L3 is a category, NOT the product.** **Batata Frita Continente** is a **MasterProduct** with **L3** = **Batata Frita Lisa**, **brand** = **Continente**. This keeps L3 useful for cross-brand €/kg comparison without losing SKU granularity.
- **Brand-as-L2 exception.** For premium-by-brand branches (**Gelados**, **Pastilhas**): L2 = brand, L3 = linha/sabor, **MasterProduct** = exact SKU. Those L2 nodes carry **brand\_axis** = **true**.
- **Empty L3 arrays are valid** placeholder L2 nodes; L3 is optional — the deepest *assigned* level is the authoritative one.
- **Invariants:** `parent.domain == child.domain`; `level == 1 ⇔ parent_id IS NULL`.
- **Seed:** `seed/supermarket/categories/supermarket-categories.pt-PT.json` supplies the vocabulary. Its `review_candidates` list documents the normalization decisions (accent/casing fixes, EN→pt-PT L1 renames **Gym→Ginásio** / **Misc→Diversos** / **Others→Outros**, merged duplicates, removed garbage tokens).

⚠ **Known stale wording in the seed file.** `exclusion_flag.meaning_en` still says `total_cents / fs_total_cents`. That predates both the decimal-EUR rule and Decision #31 — there is no stored Fs total at all now, and money is **NUMERIC(10,2)** EUR. Treat the JSON as authoritative for *vocabulary only*.

## Category Governance — editing a live tree

The taxonomy is the household's, and it will change. Each operation has a defined blast radius; all four are audited and available to **OWNER/MEMBER**:

| Operation                                              | Effect on existing data                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rename</b><br>( <code>display_name_pt</code> )      | <b>None.</b> Rows reference categories by <code>id</code> , so a rename propagates everywhere at no cost — no migration, no backfill. This is why the <code>id/display</code> split exists                                                                                                                  |
| <b>Reparent</b> (move a node under a different parent) | The deepest assigned <code>id</code> is unchanged; the <b>maintained ancestors</b> ( <code>category_l1_id</code> / <code>_l2_id</code> ) are recomputed on every affected <b>product</b> in one audited transaction — thousands of rows, not the millions of receipt lines the old shape would have touched |
| <b>Merge</b> (fold A into B)                           | Every row pointing at A is reassigned to B, then A is soft-deleted. Reassignment is audited row-by-row and is reversible from <b>AuditLog</b>                                                                                                                                                               |
| <b>Retire</b> (soft-delete)                            | <b>Blocked while any row references it.</b> The UI offers merge-into-target as the way out; a category in use is never silently orphaned                                                                                                                                                                    |

**Why ancestors are maintained rather than snapshot.** Storing the full triple as a historical snapshot would keep old rows stable but let them drift from the current tree, so "spend by L2" would silently mix two taxonomies. Deriving ancestors on every read would cost a recursive join on the hottest query in the module. The middle path — deepest level authoritative, ancestors denormalized and refreshed on reparent — gives consistent analytics and single-table queries. See Decision #16.

## Computed & Derived Fields

Evaluated on read; nothing is persisted except the immutable **ProductPriceHistory** snapshot.

- **weight\_kg** (per item) = **COALESCE(weight\_observed\_kg, weight\_listed\_kg)**
- **price\_per\_kg\_pvp\_eur** = **unit\_price\_pvp\_eur / weight\_kg** — legacy **Preco/Kg/real**, a misnomer: it is the **list** price per kg
- **price\_per\_kg\_promo\_eur** = **(unit\_price\_pvp\_eur - promo\_discount\_eur) / weight\_kg** — after the individual promo only
- **price\_per\_kg\_final\_eur** = **paid\_price\_eur / weight\_kg** — after the prorated invoice discount too
- All three are **NULL when weight\_kg is NULL**, with a stated reason — never defaulted to zero
- **subtotal\_eur** (per receipt) = **total\_eur + total\_discount\_eur** — the pre-discount figure. Note the **direction**: subtotal is *larger* than the total, because the discount has not been taken off yet
- **computed\_total\_eur** (per receipt) =  $\sum$  **paid\_price\_eur** over **every** item — Fs rows contribute **0.00** and refunds contribute negative, exactly as the receipt printed them
- **notional\_value\_eur** (per item) = **is\_fs ? unit\_price\_pvp\_eur × quantity : paid\_price\_eur**
- **notional\_total\_eur** (per receipt) = **total\_eur + fs\_value\_eur** — **what the shop would have cost had every article been paid for**
- **fs\_value\_eur** (per receipt, or any period) =  $\sum$  **notional\_value\_eur** over Fs items
- **fs\_item\_count** (per receipt) = **COUNT(items WHERE is\_fs)** — how many Fs articles were appended
- **fs\_share\_pct** = **fs\_value\_eur / total\_eur × 100** — Fs value **against what was actually paid**, so two €5 Fs articles on a €10 invoice read as **100 %**. **NULL** when **total\_eur = 0**
- **printed\_item\_count** (per receipt) = **COUNT(items WHERE NOT is\_fs)** — the figure compared against the printed **item\_count**
- **refund\_item\_count** (per receipt) = **COUNT(items WHERE quantity < 0)**
- **is\_return** (per receipt) = every item has **quantity < 0**
- **is\_reconciled** = **computed\_total\_eur** matches **total\_eur** within **receipts.arithmetic\_tolerance\_eur**
- **invoice\_discount\_ratio** = **total\_discount\_eur /  $\sum$ (pvp - promo)** over non-Fs items
- **margin\_signal** (per product/merchant) = **(current\_weight / avg\_weight\_12m) - (current\_price / avg\_price\_12m)**
- **is\_complete** (per receipt) = every item has resolved to a **master\_product\_id**
- **validated\_share** (per receipt) = share of items whose product's **category\_status** is not **AUTO**
- **last\_pvp\_eur / last\_price\_per\_kg\_eur / last\_weight\_kg / last\_observed\_on** (per product) = the newest **ProductPriceHistory** row — what pre-fills manual entry and Fs valuation

If sorting or filtering by €/kg becomes a hot path, promote it to a Postgres **GENERATED ALWAYS AS ... STORED** column rather than persisting it in application code — the derivation stays in one place.

## Functional Requirements

- **FR-1.1 OCR Ingestion.** Parse uploaded PDFs/images to extract merchant, date/time, line items, quantity/weight + unit, PVP, promotions, discounts, totals and payment-method breakdown. Persist `raw_ocr_payload` and per-field confidence. Runs as a Celery task keyed by a `ProcessingJob.idempotency_key`; a failure is a **FAILED** job row, never a silent loss.
- **FR-1.2 Manual Fallback & Editing.** Full create/edit of receipts and items, including **adding an Fs article that was never on the invoice** — classified like any other product and given a notional value. Any edit recomputes totals, IVA and €/kg, then re-scores confidence. Corrections feed the learned alias table.
- **FR-1.3 Hierarchical Categorization.** A category is assigned **to the MasterProduct, once**, and every receipt line inherits it by resolving to that product. On a brand-new product the category is auto-suggested by a **local classifier over the L3 vocabulary** fed by the description *and* the `merchant_section` heading the line sat under, then confirmed through an autocomplete picker showing the full `L1 > L2 > L3` path. `category_status` (`AUTO` | `VALIDATED` | `MANUAL`) records whether a human has checked it — so the backlog is "products never checked", a list that shrinks, not "lines never checked", a list that grows forever.
- **FR-1.4 Price Evolution & Shrinkflation.** Maintain `ProductPriceHistory`; compute `margin_signal`; render €/kg list and paid trend lines; alert on shrinkflation. Fs observations participate like any other, and every one of these views honours the `fs` filter.
- **FR-1.5 Contextual Flags & Tags.** Receipt- and item-level `tags[]` (`#férias`, `#culinária`, `#social`) are open user labels with **no effect on any total** — structurally distinct from `is_fs`, which says the article was never on the invoice, and from `product_flag`, which is read off the document. Dietary and allergen attributes are queryable per item.
- **FR-1.6 Document Storage & the Fiscal QR.** Reuse the shared `Document` pipeline. **New work:** read the Portuguese **ATCUD fiscal QR** (Portaria 195/2020), which encodes *invoice-level* fields only — issuer NIF, buyer NIF, document type, status and date, unique document ID, ATCUD, the VAT breakdown per rate, and the gross total. It carries **no line items**. Treat it as the module's **highest-confidence anchor**: when present and structurally valid, its NIF resolves the merchant exactly, its date sets `purchase_date`, and its total and VAT breakdown become the reference values that item-level arithmetic reconciles *against*. Record `atcud_valid` and `atcud_reason`. A missing or malformed QR lowers confidence and flags for review, but never blocks ingestion.
- **FR-1.7 Merchant Management.** Extend the existing `/api/merchants` router with receipt-specific needs (logo upload, per-merchant parser hints, alias management). NIF validation already exists. Receipts may only reference an existing merchant.
- **FR-1.8 Fuzzy Product Normalization.** Resolve merchant descriptions to a `MasterProduct` via **normalize-then-match**: strip accents, casing, units and pack affixes → `rapidfuzz` token-set ratio → learned `ProductAlias`. Confirmed corrections append to the alias table.
- **FR-1.9 Duplicate Prevention.** The **ATCUD is the fiscal document identity**, so a duplicate is prevented rather than detected: `(entity_id, atcud_code)` is unique, and re-uploading the same document SHA-256 is idempotent. Either collision returns the receipt already held instead of creating a second. Only when a receipt carries **no** ATCUD — an unreadable photo, a hand-written slip, a legacy import row — does a soft check on merchant + date + total warn the user before saving.
- **FR-1.10 Multi-Payment & Loyalty.** Parse composite payments (cash + card + loyalty). Record the scheme, masked card, euros accrued and euros spent **on the Receipt**, and prorate the cartão

discount into `invoice_allocated_discount_eur` per item, kept strictly separate from per-item promos. Fs articles never receive an allocation — they were not on the invoice being discounted.

- **FR-1.11 Deposits & Refunds.** Model bottle deposits (*tara*) and packaging refunds as `product_flag = DEPOSIT_RETURN`; refunds carry negative quantities and are counted by derivation, never by a stored counter.
- **FR-1.12 Bulk Weigh & Unit Ambiguity.** Distinguish listed weight (label) from observed weight (register scale); handle items priced only at checkout. Where the receipt prints €/kg outright, capture it. A missing weight suppresses €/kg with a stated reason rather than guessing one.
- **FR-1.13 Unit Standardization.** Normalize every quantity into `(quantity_canonical, unit_canonical ∈ KG|L|UN)` so cross-unit and cross-merchant comparison is well defined. Mass → kg, volume → L, count → un; never convert between the three families. Real receipts spell the same unit several ways (**KG** and **KIL** on one Piquete talão), so normalization happens at parse time.
- **FR-1.14 Reconciliation to the Ledger.** Propose `Link(RECEIPT_TRANSACTION)` edges to M2 transactions within a configurable date window and amount tolerance, scored and explained. Above the cutoff the link is auto-created as **CONFIRMED**; otherwise it becomes a **ReviewTask**. Manual linking reuses the shared transaction picker.
- **FR-1.15 Merchant Parser Profiles.** Maintain a registry of parsers keyed by merchant plus **one generic fallback**. Detection runs **before** extraction: identify the merchant from the document, select the highest-priority matching active profile, then parse with it. An unmatched document falls through to the generic profile — never to an error. Profiles are CRUD-able, can be disabled without deletion, and expose their observed `success_rate`.
- **FR-1.16 Parsing Queue & Batch Ingestion.** Accept many invoices in one upload. Each becomes its own **ProcessingJob** and **Receipt**, processed independently so one bad scan never blocks the batch. Expose a queue view over job state (**QUEUED/RUNNING/SUCCEEDED/FAILED/RETRYING**) with per-job progress, the failure reason, and **retry from the stored document** — never a re-upload.
- **FR-1.17 Category Administration.** Create, rename, reparent, merge and retire categories in the **GROCERY** domain, with the impact rules in Category Governance enforced and audited. Retire is blocked while rows reference the node; merge is the supported way out.
- **FR-1.18 Line-Level Explorer.** Browse every purchased article across all invoices as one flat, filterable, paginated grid — the shape the household already thinks in — with a one-click jump from any line to the original invoice image and to the product's full purchase history.
- **FR-1.19 Legacy Spreadsheet Import.** Import the **SUPERMARKET\_YYYY** sheets per the Source-of-Truth Mapping, wrapped in an **ImportBatch** and **re-runnable without duplication**. Fs rows carry a ~1 s timestamp nudge that must be **undone before grouping** — snap each onto the nearest non-Fs timestamp within 2 s, then group by `(Supermercado, timestamp)`. **Never group by Preço Fatura**: it is a **SUMIF** over `Full_Date`, so on a nudged Fs row it sums only itself. **PromoGlob** is recomputed deterministically, non-grocery merchants are skipped and counted, and every distinct description becomes a **MasterProduct** + **ProductAlias**. **The sheet is validated, not trusted**: each imported receipt is scored and non-reconciling groups go to **NEEDS\_REVIEW**. This is also what solves cold start (Decision #39), so it is not an epilogue but the thing that makes matching and categorization work at all.

---

## Automation Rules

- **Merchant resolution:** runs **first**, because it selects the parser. Priority is **printed NIF → alias → fuzzy name**. A receipt may carry **two** NIFs — the merchant's and the household's — and the Portuguese

numbering plan tells them apart deterministically: **1, 2, 3 are natural persons; 5, 6, 8, 9 are organisations**. Take the organisational NIF, and prefer the header block when both are organisational. Checksum-validate, then look up `Merchant.nif`. Fall back to alias/name fuzzy match — auto-accept  $\geq 0.78$ , review band 0.70–0.78, manual below. A NIF that validates but matches **no** merchant, or a *different* merchant than the name suggests, emits `merchant_nif_mismatch` and sends the receipt to review; it is never silently overridden.

- **Parser selection:** highest-`priority` active profile whose `detection_patterns` match and whose `document_kinds` include this input; otherwise the generic profile. The chosen profile is recorded on `Receipt.parser_profile_id` and shown in the UI.
- **Product resolution priority:** `ProductAlias` exact match on `description_norm` → normalized fuzzy  $\geq 0.78$  (review band 0.70–0.78) → new-product prompt. **There is no exact key** — real receipts print neither an EAN nor an article code, only a truncated description (Decision #23). `ProductAlias` is therefore not an optimisation but the load-bearing mechanism of the whole module.
- **Fs rule (CRITICAL).** An Fs article is **added by hand and never parsed**, so it takes no part in extraction or confidence scoring. There is exactly **one** arithmetic check,  $\sum \text{paid\_price\_eur} \approx \text{total\_eur}$  within tolerance, and Fs rows enter it contributing `0.00`. **Adding an Fs article to a receipt — even a CONFIRMED one — must leave `total_eur`, `computed_total_eur` and `is_reconciled` untouched**; a test asserts precisely this. An Fs article requires a `notional_value_eur > 0` with a recorded source, otherwise it is meaningless.
- **Arithmetic tolerance** is a `Setting (receipts.arithmetic_tolerance_eur)`, default **€0.02** per the brief. A €0.05 allowance may be granted only for a merchant known to round, and that allowance is itself a decision reason.
- **Loyalty allocation:** prorate the cartão discount into `invoice_allocated_discount_eur` per item, skipping Fs rows.
- **Shrinkflation:** over a rolling **12-month** window with  $\geq 3$  prior observations of the same `(master_product_id, merchant_id)`, alert when `margin_signal ≤ -0.05` — the pack shrank faster than the price fell. `margin_signal` is `NUMERIC(6,4)`: a score, not money, but decimal for reproducibility. Fs observations count towards the window unless the caller passes `fs = exclude`.
- **Duplicates.** Prevented at write time by `(entity_id, atcud_code)` and the document SHA-256, not detected afterwards. A receipt with no ATCUD falls back to a soft merchant + date + total warning.

## The Confidence Engine

A **pure, deterministic** scoring module — no clock reads, no randomness, no I/O:

```
def score(signals: ReceiptSignals) -> tuple[Status, Decimal,
list[DecisionReason]]
```

Each `DecisionReason` is `{rule, detail, score}` and surfaces behind the "Porquê?" affordance on every auto-classified value.

| Signal              | Weight | Source                                       |
|---------------------|--------|----------------------------------------------|
| Product match       | 0.30   | learned alias > normalized fuzzy description |
| OCR text confidence | 0.25   | per-field, from the extraction provider      |

| Signal                    | Weight | Source                           |
|---------------------------|--------|----------------------------------|
| Arithmetic reconciliation | 0.25   | $\Sigma$ items vs printed totals |
| Merchant match            | 0.20   | name/alias fuzzy score           |

Thresholds come from **Setting** (`confidence.auto_accept` 0.90, `confidence.review` 0.60), per-source configurable. **Learned corrections live in `ProductAlias`, outside the pure engine.** Arithmetic reconciliation is a *signal*, not a gate: a mismatch **halves** the score and emits `arithmetic_mismatch` — it never silently discards a receipt.

### Processing Engine

The pipeline runs in a fixed order, and **merchant detection comes first** because it selects the parser:

```
detect merchant → select MerchantParserProfile (or generic)
                  → extract → normalize → resolve product → classify
                  → reconcile arithmetic → score confidence
```

Each stage is a Protocol. Every stage **must** have a local implementation that works offline and unsubscribed — that is what makes the module usable on day one. A remote implementation may be registered alongside it and selected per stage:

| Protocol                        | Local implementation (ships enabled)                                                                           | Remote alternative              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>ReceiptParser</code>      | Per-merchant classes ( <code>continente_v1</code> , <code>pingodoce_v1</code> , ...) + <code>generic_v1</code> | — layout parsing stays local    |
| <code>OcrProvider</code>        | <code>pytesseract</code> — scans and photos                                                                    | Hosted OCR, when configured     |
| <code>ExtractionProvider</code> | <code>pdfplumber</code> — digital PDFs                                                                         | LLM extraction, when configured |
| <code>SuggestionProvider</code> | <code>rapidfuzz</code> token-set + learned <code>ProductAlias</code>                                           | External product database       |
| <code>CategoryClassifier</code> | Keyword/similarity match over the L3 vocabulary                                                                | LLM classification              |

**Selection is per stage, via `Setting`, and defaults to local.** Enabling a remote engine requires a credential to be present; without one the stage silently stays local rather than failing. The UI shows which engine handled a receipt, and the engine name is recorded in `decision_reasons` so a quality change is always traceable to the switch that caused it.

**Do not treat local-only as a ceiling.** If a remote engine would materially raise the auto-accept rate, the right move is to build the adapter, measure both against the same fixtures, and record the comparison — not to assume the local engine is good enough. The default exists because there is no subscription today, not because remote processing is unwelcome.

## UI / Screens

Design intent: modern, dense-but-readable, and progressive. Secondary detail lives in modals, side sheets, popovers and collapsible sections — never in a wall of fields. Every destructive or lifecycle action is one click away with a confirm.

- **UX-1.1 Upload & Parsing Queue.** Drag-drop or camera capture, **many files at once**. A queue list shows one row per invoice with its job state, the parser profile chosen, live progress, and the failure reason when it failed — with **retry** (re-runs from the stored document) and **parse manually** as escape hatches. Finished rows link straight into review.
- **UX-1.2 Receipt Review (crown jewel).** Split pane: the original image/PDF on one side, parsed line items on the other.
  - Uncertain fields are flagged **by icon and label, never by colour alone**.
  - Inline edits recompute running totals live and warn when  $\Sigma \text{ items} \neq \text{total}$ .
  - Inline **MasterProduct autocomplete** (search-as-you-type over canonical names, brands and known aliases) with a "create new product" affordance; suggested category and allergen tags follow the pick.
  - Category picker is an **autocomplete over the tree**, showing the full **L1 > L2 > L3** path so the same leaf name under two parents is never ambiguous.
  - A **"Porquê?"** popover on every auto-filled value showing its **decision\_reasons**.
  - A single **"confirmar categorias"** action promotes every **AUTO** classification on the receipt to **VALIDATED**.
  - An **"adicionar artigo Fs"** action appends an article that was not on the invoice, with the same product and category pickers as any other line plus a notional value pre-filled from price history. Fs rows are visually segregated from the printed lines and shown with their own subtotal.
  - Optimistic updates for single-item edits only — never for aggregates.
- **UX-1.3 Receipts List (level 1 — one row per invoice).** Filterable by merchant, date range, tag, category, **fs (only | exclude | all)** and status; summary cards for total spend, Fs value and item count; sorting, pagination and a density toggle.
- **UX-1.4 Item Explorer (level 2 — one row per purchased article).** The flat view the household already thinks in: every line across every invoice, filterable by product, category, merchant, date, **fs (only | exclude | all)** and flag. Columns mirror the legacy sheet (date · merchant · product · category · qty · weight · PVP · promo · paid · €/kg). Each row links to the source invoice — opening the review pane focused on that line — and to the product's purchase history. CSV export.
- **UX-1.5 Estado (status dashboard).** The module's front door — one screen answering "what needs me?", each figure a link into the filtered list that resolves it:
  - **Faturas por processar** — **ProcessingJob** in **QUEUED/RUNNING**, plus **FAILED** ones needing a retry.
  - **Faturas por validar** — receipts in **NEEDS\_REVIEW**, split by what triggered it.
  - **Linhas por resolver** — items with no **master\_product\_id**, the count that blocks categorization.
  - **Produtos por categorizar** — products whose **category\_status** is still **AUTO**.
  - **Produtos a fundir** — near-duplicate **canonical\_name** candidates, so the catalogue does not silently fork.
  - **Taxa de auto-aceitação observada** — shown as a trend, labelled explicitly as *observed, not targeted*.

- **UX-1.6 Master Product Manager.** CRUD over canonical name, brand, **category**, **sold\_by\_weight**, **pack variants (label + weight per pack size)**, per-merchant aliases with confidence, dietary/allergen checkboxes, seasonal flags, and a merge-duplicates action. Each product shows its last known price and every receipt line it has ever appeared on.
- **UX-1.7 Category Administration.** Tree editor for the **GROCERY** domain: rename, drag-to-reparent, merge and retire. Every destructive operation shows **how many rows it will touch before it runs**, and retire is refused while the node is in use with merge offered as the alternative.
- **UX-1.8 Merchant Parser Profiles.** List of profiles with merchant, document kinds, priority, active toggle and observed **success\_rate**; edit **field\_hints**; a "test against this receipt" action that re-parses a stored document with a chosen profile and diffs the result.
- **UX-1.9 Price Evolution.** Product selector; €/kg list vs paid multi-line trend by merchant; shrinkflation overlay; an **fs** toggle (**only** | **exclude** | **all**) that redraws the series; CSV export.
- **UX-1.10 Loyalty Savings.** A view grouped by scheme and masked card number — a **GROUP BY** over **Receipt**, not a screen of its own to manage. Associated receipts, euros accrued and spent, per-receipt allocation breakdown.

## Proposed API Surface

Indicative; may be refined during implementation as long as the capabilities and rules above are preserved. Routes are namespaced **/receipts** under **/api**.

### 1. Receipts & the Parsing Queue

- **POST /receipts** — multipart upload, **one or many files**; requires an **Idempotency-Key**; returns one **ProcessingJob** per file.
- **GET /receipts/queue** — job state, progress, chosen parser profile and failure reason per queued invoice.
- **POST /receipts/{receiptId}/reparse** — re-run from the stored document, optionally forcing a **parser\_profile\_id**. Never requires re-upload.
- **GET /receipts** — filters: merchant, date range, tag, category, **fs**, status, entity.
- **GET /receipts/status** — the counts behind UX-1.5: to process, to validate, lines unresolved, products uncategorized, merge candidates, observed auto-accept rate.
- **GET /receipts/{receiptId}** — items, confidence breakdown, decision reasons and links.
- **PATCH /receipts/{receiptId} · PATCH /receipts/{receiptId}/items/{itemId}** — recompute totals and rescore.
- **POST /receipts/{receiptId}/items** — append an item that was not on the invoice (**is\_fs = true**), with its notional value and source.
- **POST /receipts/{receiptId}/confirm** — transition to **CONFIRMED**, resolve the **ReviewTask**, feed the learning loop.
- **POST /receipts/{receiptId}/confirm-categories** — promote every **AUTO** classification on the receipt to **VALIDATED**.
- **POST /receipts/{receiptId}/void** — void with a mandatory reason; replaces delete once confirmed.

### 2. Line Items (level 2)

- **GET /receipt-items** — flat, paginated, filterable by product, category, merchant, date, **fs** and flag; CSV export.
- **PATCH /receipt-items/{itemId}/product** — re-resolve a line to a different **MasterProduct**; the category follows and the correction feeds **ProductAlias**.

### 3. Products & Aliases

- **GET/POST/PATCH/DELETE /master-products** — full CRUD including **category\_l3\_id**, **sold\_by\_weight** and **pack\_variants**.
- **GET /master-products/search?q=** — **autocomplete** over canonical name, brand and known aliases; returns the category and **last known price** so any picker can pre-fill.
- **GET /master-products/{productId}/occurrences** — every receipt line this product has appeared on, with links to each invoice.
- **POST /master-products/{productId}/merge** — deduplicate master records.
- **GET /master-products/{productId}/price-history** — with shrinkflation signals; accepts the **fs** filter.
- **POST /product-aliases/learn** — update alias confidence from a user correction.

### 4. Categories

- **GET /categories/search?q=&domain=GROCERY** — **autocomplete** returning the full **L1 > L2 > L3** path.
- **POST /categories** · **PATCH /categories/{categoryId}** — create and rename.
- **POST /categories/{categoryId}/reparent** — move; recomputes maintained ancestors on affected rows.
- **POST /categories/{categoryId}/merge** — fold into a target, reassigning every referencing row.
- **DELETE /categories/{categoryId}** — retire; **refused while in use**, with the usage count returned.
- **GET /categories/{categoryId}/impact** — how many receipt items and master products a destructive operation would touch, **before** it runs.

### 5. Parser Profiles

- **GET/POST/PATCH/DELETE /parser-profiles** — CRUD; the generic profile cannot be deleted.
- **POST /parser-profiles/{profileId}/test** — re-parse a stored document with this profile and return a diff against the current result.

### 6. Analytics & Import

- **GET /receipts/analytics/shrinkflation** · **GET /receipts/analytics/category-spend** · **GET /receipts/analytics/loyalty** — all accept the **fs** filter.
- **POST /receipts/import/legacy** — the Excel migration, wrapped in an **ImportBatch**.

**Reused, not rebuilt:** the Core **Merchant**, **Tag**, **Document** and **ReviewTask** surfaces, and the shared transaction picker. Category **reads** are Core; the administration endpoints above extend that router and are scoped to **domain = GROCERY**.

## Analytics & KPIs

- **Ingestion quality, reported not targeted:** observed auto-accept rate, share of lines resolved, share of products categorized — each as a trend over time so progress is visible without a threshold to game.
- €/kg evolution (list and paid) per product and merchant, including seasonal variance.
- Shrinkflation: products whose weight or unit count fell against the trailing 12 months.
- Loyalty savings: cumulative discount per receipt, per scheme and masked card number.
- **Two measures, one grammar.** Every analytic is expressible over `paid_price_eur` ("what did I spend") or `notional_value_eur` ("what was it worth"), and the two are **identical on non-Fs rows** — which is what lets Fs articles join any dashboard without a special case. Combined with the `fs` filter (`only` | `exclude` | `all`, default `all`), the household can ask "what did I pay?", "what was I given?" or "what did I consume?" from the same query.
- Fs budget tracking: `fs_value_eur`, `fs_item_count` and `fs_share_pct` — Fs value against what was paid, per receipt and over any period.
- Basket composition: category split, dietary breakdown, organic share of spend.
- Promotion ROI: discount rate by type (BOGO, percentage, absolute, loyalty) and by merchant.
- Duplicates: prevented at write time; the count of collisions rejected.

---

## Performance

The NFR is **<800 ms p95 on a 10-year seed**, measured with `EXPLAIN (ANALYZE, BUFFERS)`, never assumed.

- `receipts (entity_id, purchase_date DESC, id)` — the shape of every list and dashboard query.
- `receipt_items (receipt_id, line_no)`, `receipt_items (master_product_id)` — category spend joins through the product, which is a small table.
- `master_products (category_l1_id, category_l2_id, category_l3_id)`.
- `product_price_history (master_product_id, merchant_id, observed_on DESC)` — `is_fs` lives on the row so the Fs filter never forces a join back to `receipt_items`.
- `documents (sha256_hash)` — already exists; with `receipts (entity_id, atcud_code)` it is what makes a duplicate impossible.
- Dashboards read **materialized monthly aggregates** plus a Redis cache invalidated on confirm. `raw_ocr_payload` is never selected in list queries.

---

## Edge Cases & Business Rules

- **Reconciliation.**  $\sum \text{paid\_price\_eur} == \text{total\_eur} \pm \text{tolerance}$  (default €0.02) over every item; Fs rows enter at `0.00` and are excluded from the printed `item_count` check.
- **Multi-weight units.** kg-priced and unit-priced items are handled separately; an item with both a count and a weight (3 packs × 250 g) carries both and normalizes to kg.
- **Missing prices.** Bulk-weighed items without an upfront price are flagged for manual entry; €/kg stays `NULL` with a stated reason.
- **OCR garbling.** Thermal receipts corrupt €, ¢, ¢. Normalization strips diacritics before matching; unrecoverable text lowers confidence rather than guessing.

- **Refunds & returns.** Negative quantities are refunds — excluded from the paid total, surfaced through the derived `refund_item_count`, and they may link back to the original purchase. A receipt whose items are all negative derives as `is_return`.
- **Deposits (tara).** Never categorized as groceries; reconciled when the refund is credited.
- **Payment ambiguity.** "DINHEIRO + CARTÃO" without a printed split prompts the user rather than assuming an allocation.
- **Missing ATCUD or NIF.** Flags for review with a tax-compliance warning; never blocks ingestion. A NIF that validates but matches no known merchant raises `merchant_nif_mismatch` rather than guessing.
- **IVA.** Not modelled (Decision #38). The printed token is kept in `iva_class_raw` and interpreted by nothing.
- **Idempotent re-import.** The same document SHA-256 within the same entity never creates a second receipt.
- **Entity attribution.** A receipt belongs to exactly one entity. With the selector on «todas» the write is **refused, not guessed**; the upload dialog asks for the entity inline.
- **Deletion.** Confirmed receipts are **voided with a reason**, never deleted. Soft delete applies before confirmation.

## Deferred (explicitly not built now)

- **A bundled remote engine.** The provider seam, the per-stage `Setting` and the credential check all ship; what does not ship is a configured cloud OCR, hosted LLM or external product-database adapter. Adding one is ordinary follow-up work, not a redesign — see Decision #13.
- **Barcode scanning.** Scanning a product package to look up or create a `MasterProduct`, and any EAN check-digit validation that would come with it. `pack_variants[].barcode` is the slot it would populate. Receipts never supply one, so nothing today would read it.
- **ATCUD digital-signature** verification (the QR is read and structurally validated in v1; the cryptographic hash chain is not verified).
- SKU batch/lot tracking and product-recall alerts.
- Return-window and freshness alerts from `expected_shelf_life_days` (the field is stored, the alerting is not built).
- Bulk-vs-retail price-anomaly flagging.
- Seasonal-produce anomaly analytics (`seasonal_flags` is stored and filterable; the model is deferred).
- Hand-written receipt OCR tuning — supported, but routed straight to manual review.
- Multi-merchant mall receipts as a single payment transaction (see Decision #7).
- Data-driven parser authoring (building a new merchant parser from the UI alone). v1 registers parsers in code and tunes them with `field_hints`.

## Definition of Done

- ☐ FR-1.1 – FR-1.19 implemented **across the three delivery slices**, each leaving `./fm check` green and a working stack from clean.
- ☐ **Every line and every receipt shows a confidence**, and the UX-1.5 status screen reports what is waiting: faturas por processar, por validar, linhas por resolver, produtos por categorizar e a fundir. Each figure links to the list that resolves it.

- ☐ The observed auto-accept rate is displayed as a trend and **labelled as observed, not targeted** (Decision #41).
- ☐ All four merchant profiles parse their own fixtures correctly, including the **opposite discount semantics** (Continente  $\Sigma \text{ lines} - \text{cartão} = \text{total}$ , Pingo Doce  $\Sigma \text{ lines} - \text{poupança} = \text{total}$ ) and the positional differences (IVA token first at Continente, last at Lidl, quantity first at Piquete).
- ☐ Extracted word boxes are committed as **golden files** for all eleven fixtures, so an extractor upgrade and a parser regression are distinguishable.
- ☐ The Piquete photograph parses end to end through OCR, proving the image path works on a skewed, partly obscured *talão*.
- ☐ A product's **last known price** pre-fills manual entry — pack price for packaged goods, €/kg for **sold\_by\_weight** ones.
- ☐ The **iva\_class\_raw** token is captured verbatim where printed, and nothing downstream reads it.
- ☐ Merchant NIF is taken from the header block — a fixture carrying **both** merchant and buyer NIFs resolves to the merchant.
- ☐ **merchant\_section** is captured for every line and measurably improves classification of unseen products.
- ☐ **The full pipeline completes with egress blocked, at default settings** — proving the module needs no subscription. The provider seam is exercised by a fake remote engine in tests, proving a stage can be swapped without touching the pipeline.
- ☐ A merchant-specific parser and the generic fallback both parse their fixtures; a document matching no profile still parses generically rather than failing.
- ☐ Batch upload of N invoices creates N independent jobs; one deliberately corrupt file fails alone and is retryable from the stored document without re-upload.
- ☐ **Fs proven by unit tests:** adding an Fs article to a **CONFIRMED** receipt leaves **total\_eur**, **computed\_total\_eur**, **is\_reconciled** and the printed-count check **unchanged**, while **notional\_total\_eur** rises by its value; the €10 invoice + 2×€5 Fs case yields **notional\_total\_eur = 20.00** and **fs\_share\_pct = 100**.
- ☐ An Fs article writes a **ProductPriceHistory** observation carrying its notional value in **both** price columns, marked **is\_fs**, and never **0.00**.
- ☐ Fs articles appear in consumption, quantity and price history like any other product, and every such query — including €/kg trends and shrinkflation — honours the **fs** filter.
- ☐ The printed **item\_count** check is unaffected by Fs articles, and **fs\_item\_count** reports them separately.
- ☐ Prorated invoice discount proven: summing per-item promos alone does **not** reconcile; only **pvp - promo - invoice\_allocated** does.
- ☐ All three €/kg variants computed and distinct on a fixture carrying both a per-item promo and an invoice-level discount.
- ☐ Confidence engine is pure: identical input yields byte-identical (**status**, **confidence**, **decision\_reasons**) across runs.
- ☐ Every auto-decision exposes human-readable reasons in the UI and is user-overridable; corrections update **ProductAlias**.
- ☐ Every line resolves to a **MasterProduct** and inherits its category; a brand-new product arrives with an **AUTO** suggestion that a human can promote to **VALIDATED** once, for every past and future line.

- ☐ Category rename leaves every referencing row untouched; reparent recomputes maintained ancestors on all affected rows in one audited transaction; retire is refused while in use and merge reassigns and audits.
- ☐ A product with two pack sizes of different weights resolves correctly from the size token in the description and yields comparable €/kg.
- ☐ Both browse levels work: one row per invoice, and one row per article across all invoices with a working jump to the source document.
- ☐ Receipt status machine rejects illegal transitions; confirmed receipts are voided, never deleted; every mutation writes **AuditLog**.
- ☐ Re-upload of the same file is idempotent — no duplicate receipt, no duplicate **ProcessingJob**.
- ☐ **Two receipts sharing an ATCUD cannot both exist**: the second upload returns the first. A receipt with no ATCUD instead raises a soft merchant + date + total warning.
- ☐ IVA inference reconciles with invoice totals within  $\pm\text{€}0.01$ .
- ☐ ATCUD fiscal QR read and structurally validated; its NIF, date, total and VAT breakdown used as the reconciliation reference when present; a missing or malformed QR flags for review without blocking.
- ☐ Receipt↔transaction **Link** suggestions created above the cutoff, reviewed below it; manual linking uses the shared transaction picker.
- ☐ **The 2025 legacy import runs end to end**: Fs timestamps are snapped back, 2,429 rows group into **288 receipts** by (**Supermercado**, **timestamp**),  $\geq 274$  reconcile within  $\text{€}0.02$ , the rest land in **NEEDS\_REVIEW** with reasons, non-grocery merchants are skipped and counted, and  $\sim 1,564$  products with aliases exist afterwards. Re-running duplicates nothing.
- ☐ **No more than 6 all-Fs groups survive** — a regression above that means the snap window or the grouping key has broken and Fs articles are being torn from their invoices.
- ☐ The two Pingo Doce  $\text{€}1,19$  rows of 25/01, five hours apart, import as **two receipts, not one**.
- ☐ The 425 **F/f** rows import as Fs articles with **paid\_price\_eur = 0.00**, leaving every **total\_eur** untouched.
- ☐ Merchant find-or-create folds **Mercadona/mercadona** into one merchant across all 18 legacy spellings.
- ☐ Dashboard queries measured under 800 ms p95 on the 10-year seed with **EXPLAIN (ANALYZE, BUFFERS)**.
- ☐ Seed: realistic Portuguese receipts including appended Fs articles, a loyalty discount, a refund and a deposit return.
- ☐ **make check** green; **docs/** and the root **README.md** updated in the same change.

**Testing discipline (brief §7 — this supersedes any coverage target).** Do **not** chase a coverage percentage.

Test only what is load-bearing and easy to get subtly wrong: the Fs valuation rules, the prorated-discount arithmetic, the three €/kg variants, decimal money handling, the pure confidence-scoring functions, idempotent re-import and retry, and the receipt status-machine guards. Everything else is verified by exploratory testing against this Definition of Done. Each test file states which rubric bullet it protects.

**Note on e2e.** Playwright is **not yet set up** (**apps/web/tests/e2e/** does not exist). Upload → review → confirm is the best candidate for the first e2e test, so M1 either stands up that tooling or explicitly records the deferral — it must not silently claim e2e coverage.

## Open Questions / Decisions

| #  | Question                                                 | Decision                                                                                                                                                                                                                                                                                       |
|----|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | How are duplicates handled?                              | <b>Prevented, not detected.</b> See Decision #35 — the ATCUD is a unique fiscal identifier, so a uniqueness constraint replaces the whole fuzzy-matching apparatus                                                                                                                             |
| 2  | Items sold by count rather than weight?                  | <b>unit = UN</b> ; compute price-per-unit and leave €/kg <b>NULL</b> with an explicit reason. Never fabricate a weight                                                                                                                                                                         |
| 3  | Is Fs per item or per receipt?                           | Per item, and always added by hand. There is no receipt-level toggle — an Fs article is a row the user appends, not a flag flipped on a printed line                                                                                                                                           |
| 4  | A <b>LoyaltyCard</b> table with an encrypted number?     | <b>No.</b> Merchants print the number already masked (XXXXXXXX3394X), so nothing sensitive ever arrives. <b>Receipt.loyalty_card_masked</b> holds what was printed; "which cards have I used" is a <b>GROUP BY</b>                                                                             |
| 5  | How many corrections before an alias is trusted?         | Learn on the <b>first</b> confirmed correction — <b>confidence</b> starts low and rises with <b>correction_count</b> . Waiting for five corrections discards exactly the signal that makes matching improve; confidence, not a counter, gates auto-apply                                       |
| 6  | Hand-written receipts?                                   | Accepted, always routed to manual review — never auto-accepted                                                                                                                                                                                                                                 |
| 7  | Multi-merchant / mall receipts?                          | Out of scope for v1. One <b>Receipt</b> = one payment transaction at one merchant                                                                                                                                                                                                              |
| 8  | Does <b>ReceiptItem</b> carry its own <b>entity_id</b> ? | Yes, denormalized from the parent. Redundant, but it keeps every analytics query single-table and matches the pattern every other module uses. The receipt stays the source of truth on re-attribution                                                                                         |
| 9  | What is the arithmetic tolerance?                        | €0.02 by default (brief §3), stored in <b>Setting</b> so a rounding-prone merchant can be widened without a deploy. <b>This file previously said €0.05 — that conflict is resolved in favour of the brief</b>                                                                                  |
| 10 | Where does the receipt↔transaction link live?            | The shared <b>Link</b> table with <b>link_type = RECEIPT_TRANSACTION</b> . M1 never defines a ledger row                                                                                                                                                                                       |
| 11 | Is €/kg stored or derived?                               | <b>Derived everywhere</b> , including on <b>ProductPriceHistory</b> — price and weight sit on the same immutable row, so the quotient cannot drift. Promote to a generated column only if sorting by €/kg becomes hot                                                                          |
| 12 | Are the printed totals redundant with the item sums?     | No. The printed values are independent extracted facts; the item sums are what we parsed. Reconciling the two <b>is</b> the arithmetic confidence signal, so both must be stored                                                                                                               |
| 13 | May processing use a remote provider?                    | <b>Not by default, but the door stays open.</b> Every stage must have a local engine that works offline and unsubscribed. A remote one may be registered alongside and enabled per stage from <b>Setting</b> ; without a credential the stage silently stays local. This is a <b>default</b> , |

| #  | Question                                                              | Decision                                                                                                                                                                                                                                                                                                                                                                      |
|----|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                                                                       | <b>not a ban</b> — if a remote engine would materially improve results, build the adapter and measure both against the same fixtures                                                                                                                                                                                                                                          |
| 14 | One parser for everything, or one per merchant?                       | <b>One per merchant, plus a generic fallback.</b> Layouts differ more than they resemble each other, and a single parser regresses on merchant A whenever tuned for B. Merchant detection therefore runs <i>before</i> extraction; an unknown merchant falls through to the generic parser with lower confidence                                                              |
| 15 | Is a product the same across pack sizes?                              | <b>Yes</b> — that is what makes €/kg comparable, and it is the household's own mental model. What varies between sizes is the <b>weight</b> , so <code>pack_variants</code> holds <code>{label, weight_kg, barcode?}</code> per size. This replaces four fields with one                                                                                                      |
| 16 | What happens to existing data when the category tree changes?         | Rename costs nothing (rows reference ids). Reparent recomputes <b>maintained ancestors</b> in one audited transaction, so history is re-expressed under the new tree. Merge reassigns then soft-deletes; retire is blocked while in use. Snapshotting the full triple per row was rejected — it lets history drift and mixes two taxonomies in one report                     |
| 17 | How do we know a category was actually checked?                       | <code>MasterProduct.category_status</code> is <b>AUTO</b> until a human confirms it ( <b>VALIDATED</b> ) or assigns it directly ( <b>MANUAL</b> ). Because it sits on the product, the unchecked list is finite and shrinks as the catalogue matures, rather than growing with every shop                                                                                     |
| 18 | Where does category administration live — Core or M1?                 | The <b>Category entity</b> is Core §1a. The administration UI and the impact rules are M1's, scoped to <code>domain = GROCERY</code> , because M1 is the module that lives or dies by that tree. Other domains manage their own branches when they arrive                                                                                                                     |
| 19 | Does <b>Receipt</b> store the merchant's NIF?                         | <b>No.</b> <code>Merchant.nif</code> already exists in Core, checksum-validated; the printed NIF's job is to <i>identify</i> the merchant during parsing. Storing it again creates two truths for one fact. A disagreement is a <b>matching problem</b> — surfaced as a decision reason and sent to review, not persisted. The raw text stays in <code>raw_ocr_payload</code> |
| 20 | Is <code>subtotal_eur</code> stored?                                  | <b>No — derived</b> as <code>total_eur + total_discount_eur</code> . Two of the three printed figures determine the third                                                                                                                                                                                                                                                     |
| 21 | Are <code>is_return</code> and <code>refund_item_count</code> stored? | <b>No — derived.</b> Neither is printed, so neither is an independent fact. Refunds work through negative quantities                                                                                                                                                                                                                                                          |
| 22 | Do Fs articles belong in <code>tags[]</code> ?                        | <b>No.</b> <code>is_fs</code> is structural — it says the article was never on the invoice and is valued notionally; <code>tags[]</code> are open labels with no effect on arithmetic. An article can be <b>Fs</b> and tagged <code>#férias</code>                                                                                                                            |
| 23 | Does a receipt line carry a barcode or article code?                  | <b>Neither — confirmed across eleven fixtures from four merchants.</b> A <i>talão</i> prints an IVA token, a truncated description and a value. So                                                                                                                                                                                                                            |

| #  | Question                                       | Decision                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                                                | <b>ProductAlias</b> and normalized fuzzy matching are the <b>only</b> way a line ever resolves — load-bearing, not an optimisation. <b>barcode</b> survives only as an optional slot in <b>pack_variants</b> for the deferred scan flow                                                                                                                                                                                                                                                                                       |
| 24 | What is the fiscal QR actually worth?          | Far more than validation. It encodes the issuer NIF, document date, gross total and VAT breakdown, making it the <b>highest-confidence anchor in the module</b> . The ATCUD is also printed as plain text ( <b>ATCUD:JFP767JJ-035904</b> ), so a QR that will not decode still yields the reference                                                                                                                                                                                                                           |
| 25 | How is size determined without a code?         | By parsing the description, which is where the till puts it ( <b>POLPA TOMATE GULOSO **500G**, SAL GROSSO CONTINENTE **1KG**</b> ). <b>pack_variants</b> supplies the candidate weights and the parsed token picks between them                                                                                                                                                                                                                                                                                               |
| 26 | Is the IVA rate inferred?                      | <b>Superseded by #38 — IVA is not modelled at all</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 27 | Do receipts help with categorization?          | <b>Yes, at no extra cost.</b> All four merchants group lines under their own headings ( <b>Mercearia Salgada, FRUTAS E VEGETAIS, -- TUBERCULOS --</b> ). Stored as <b>merchant_section</b> and fed to the classifier, this is a strong prior on a brand-new product                                                                                                                                                                                                                                                           |
| 28 | Do all merchants treat discounts the same way? | <b>No, and they are opposite.</b> Continente's <b>SUBTOTAL</b> is already net of <b>POUPANCA</b> , only the cartão subtracted ( <b>40,56 - 4,00 = 36,56</b> ); Pingo Doce's <b>TOTAL</b> is gross and <b>POUPANÇA</b> is subtracted ( <b>15,70 - 0,50 = 15,20</b> ). One parser would silently overstate one household's spending. <b>field_hints.line_value_is_net</b> per profile                                                                                                                                           |
| 29 | Which NIF is the merchant's?                   | The <b>organisational</b> one, decided by its first digit: <b>1/2/3</b> are natural persons, <b>5/6/8/9</b> are organisations. Continente prints its own <b>NIF: PT501591109</b> and the household's <b>NIF: PT209362367</b> ; the leading <b>5</b> versus <b>2</b> separates them with no positional guessing. Header position is the tie-break when both are organisational                                                                                                                                                 |
| 30 | Points or euros?                               | <b>Euros.</b> <b>ACUMULOU NO SEU CARTAO 4,06€</b> and <b>Combustível ganho na compra: 0 EUR</b> . <b>Receipt.loyalty_accrued_eur</b> and <b>loyalty_discount_eur</b> record them; there are no points. Continente's separate <i>selos</i> collectible scheme is out of scope                                                                                                                                                                                                                                                  |
| 31 | What exactly is an Fs article?                 | <b>An article that was never on the invoice</b> , appended by hand — not a printed line reclassified. Hence no <b>line_no</b> , <b>merchant_section</b> or <b>iva_class_raw</b> , exclusion from the printed <b>item_count</b> check, and the guarantee that adding one disturbs no reconciliation: <b>paid_price_eur</b> is <b>0.00</b> . <b>notional_total_eur = total_eur + fs_value_eur</b> ; <b>fs_share_pct</b> measures Fs <b>against what was paid</b> — €10 of Fs on a €10 invoice is <b>100 %</b> , not <b>50 %</b> |

| #  | Question                                                             | Decision                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 | Is <code>is_fs</code> part of <code>product_flag</code> ?            | <b>No, separate.</b> Every <code>product_flag</code> value is read off the document; an Fs article has no document origin at all, and an appended article can still be <code>SEASONAL</code>                                                                                                                                                                                                                                                                                                                                                    |
| 33 | Do Fs estimates pollute price history?                               | <b>No — trusted and included</b> , carrying <code>is_fs</code> onto the snapshot so every price view can filter. The one guard is arithmetic: an Fs snapshot stores the notional value in <b>both</b> price columns, because a literal <code>0.00</code> would pull paid-price trends toward zero                                                                                                                                                                                                                                               |
| 34 | Category on <code>ReceiptItem</code> or <code>MasterProduct</code> ? | <b>On the product.</b> The module's own rule settles it — <i>"two items with different categories are different master products"</i> — so a per-line category could only drift from it. Consequences: reparent rewrites thousands of product rows not millions of line rows; a category is confirmed <b>once per product</b> , so the backlog shrinks instead of growing with every shop; and an Fs article inherits its category by resolving. Category spend costs one indexed join                                                           |
| 35 | A <code>ReceiptDuplicateLog</code> table?                            | <b>No.</b> The ATCUD is a legally unique document id, so duplicates are made <b>impossible</b> by <code>UNIQUE (entity_id, atcud_code)</code> plus SHA-256 idempotency — not detected by fuzzy scoring. A receipt with no readable ATCUD gets a soft merchant + date + total warning                                                                                                                                                                                                                                                            |
| 36 | Is <code>F</code> safe to pattern-match?                             | <b>No — it is overloaded three ways.</b> Lidl uses <code>F</code> as an <b>IVA class letter for 0 %</b> ( <code>Deposito 0.10 0,10 F</code> ); every merchant numbers documents <code>FS ...</code> for <i>fatura simplificada</i> ; and the household calls its added articles "Fs". Only the third is ours, and it is never printed. A parser that greps for <code>F</code> would tag deposits as freebies                                                                                                                                    |
| 37 | Is <code>LoyaltyAllocation</code> a table?                           | <b>No — four nullable fields on <code>Receipt</code>.</b> It was strictly one row per receipt, and only one merchant of four prints a scheme at all. The limit is honest: this holds <b>one monetary scheme per receipt</b> ; a second would make it a table again                                                                                                                                                                                                                                                                              |
| 38 | Is IVA modelled?                                                     | <b>No — the largest simplification available.</b> Nothing consumed it: no KPI, no deduction, no filing. It cost a per-merchant map because <b>the same letter means different rates</b> ( <code>A</code> is 6 % at Continente, 23 % at Lidl), a token that is a letter at three merchants and a percentage at the fourth, a collision with the Fs vocabulary, and an inference rule — all to buy a check that <code>Σ lines ≈ total</code> already performs. <code>iva_class_raw</code> keeps the printed token, so a backfill could recover it |
| 39 | What happens when the catalogue is empty?                            | <b>Nothing resolves, and that is correct.</b> A cold first pass sends everything to review. The fix is data, not thresholds: the legacy import seeds ~1,564 products and their aliases, which is why it sits in slice M1b rather than at the end                                                                                                                                                                                                                                                                                                |

| #  | Question                                            | Decision                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 40 | Are this file's transcriptions a parsing benchmark? | <b>No.</b> They came from <b>pdftotext -layout</b> and a vision model; production uses <b>pdfplumber</b> word boxes and <b>pytesseract</b> , and the photograph will parse far worse. See Extraction Strategy                                                                                                                                                                                                                                                                               |
| 41 | Is there an $\geq 80\%$ auto-accept target?         | <b>No — dropped.</b> It was invented before anything was measured and was quietly corrosive: the cheapest way to hit it is to lower <b>confidence.auto_accept</b> , raising the number while lowering quality. It also needed an elaborate ritual to mean anything — second pass, PDFs only, post-import — a sign the metric served itself. Replaced by <b>confidence on every line and receipt</b> plus the UX-1.5 status screen. The rate is still shown, <b>observed, never targeted</b> |

### Source-of-Truth Mapping (legacy Excel → model)

The household's **SUPERMARKET\_YYYY** sheets are the migration source; the real **2025 file** holds **2,429 rows across 288 receipts and 18 distinct merchant spellings**. The sheet is **flat**: it has no **Receipt** parent and derives the invoice total with **SUMIF**. Everything below was measured against that file, not assumed.

⚠

**Undo the Fs timestamp nudge first, then group by the timestamp itself.** Fs rows were recorded about **a second after** the invoice they belong to — an artefact of how the sheet was filled in, not a fact about the purchase. Measured: **401 of 425 Fs rows sit at exactly +1 s** from the nearest earlier non-Fs row of the same shop and day; 17 sit at +0 s.

That one second did real damage, because the sheet's **Preço Fatura** is a **SUMIF** over **Full\_Date**: a nudged Fs row **summed only itself**, so its "invoice total" is its own price — true of 59 rows outright. Grouping on that column therefore invents a separate one-line invoice for the Fs article, which is the exact opposite of the association Decision #31 exists to preserve.

**The rule:** for each Fs row, snap its timestamp back onto the nearest non-Fs timestamp within **2 seconds** on the same shop and day; then group by **(Supermercado, timestamp)**. Snapping beats blanket **−1 s** because 17 rows were never nudged. No date-only key, no gap heuristic, no **Preço Fatura** in the key.

| Grouping rule                                             | Receipts   | Groups that are only Fs | Reconciling                               |
|-----------------------------------------------------------|------------|-------------------------|-------------------------------------------|
| <b>(shop, day, Preço Fatura)</b>                          | 431        | <b>150</b> — the bug    | 427, inflated by 150 trivial self-matches |
| Blanket <b>−1 s</b> on every Fs row                       | 295        | 13                      | 274                                       |
| <b>Snap <math>\leq 2</math> s, then (shop, timestamp)</b> | <b>288</b> | <b>6</b>                | <b>274</b>                                |

288 receipts over 2,429 rows is  $\sim 8.4$  articles per shop, which is what a grocery receipt actually looks like; the 431 figure was  $\sim 5.6$  and was wrong. **274 reconcile within €0.02; the other 14 go to review.** The 6 surviving

all-Fs groups are Fs articles with no invoice to attach to — import them as receipts with `total_eur = 0.00` and flag them, rather than inventing a parent.

**The import validates rather than trusts.** The sheet was maintained by hand over years and is wrong in places, so every imported receipt is scored like a parsed one: it gets a `confidence` and `decision_reasons`, and a group that does not reconcile lands in `NEEDS_REVIEW` instead of being written as fact. At minimum the importer checks that  $\sum \text{Price\_Final}$  over non-Fs rows matches the invoice total, that  $\text{Price\_Final} \approx \text{Price} - \text{PromoInd} - \text{PromoGlob}$ , that dates and merchants are plausible, and that every Fs row found an anchor — reporting each exception rather than silently repairing it.

**Merchant names need normalizing before find-or-create.** `Mercadona` and `mercadona` are the same shop; `Fruta`, `Fruta Chines`, `Super Fruta King` and `Loja chines` are near-collisions. Match case-insensitively on a trimmed name.

**Non-grocery merchants are skipped for now.** IKEA, Leroy, Wells, Normal, Action and El Corte Inglés are ~5 % of rows and fall outside this module's scope. The importer **excludes them and reports the count**, so the decision stays visible and reversible — widening the scope later is a re-run, not a migration.

**Fs is not an edge case: 425 rows, 17.5 % of the sheet** (F 419, lowercase f 6 — match case-insensitively). `Preço Fatura` on those rows already excludes their value, which independently confirms the model in Decision #31.

| Legacy column                                                                         | Model target                                                                                                                                | Notes                                                                                                                                                                                |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Full_Date</code> ( <code>Date + Hora</code> ) — Fs rows snapped back $\leq 2$ s | the <b>grouping key</b> , with <code>Supermercado</code> → one <code>Receipt</code> ; and <code>Receipt.purchased_at + purchase_date</code> | Verified: 288 groups, 274 reconciling. Keep the Lisbon calendar date                                                                                                                 |
| <code>Date_ID</code>                                                                  | <i>nothing</i>                                                                                                                              | Day granularity only; using it as a key merges two visits with the same total — e.g. two Pingo Doce €1,19 rows on 25/01 that are 5 h apart                                           |
| <code>Supermercado</code>                                                             | <code>Receipt.merchant_id</code>                                                                                                            | Find-or-create on a <b>trimmed, case-folded</b> name across all 18 spellings                                                                                                         |
| <code>Descrição</code>                                                                | <code>ReceiptItem.description_raw</code> → <code>description_norm</code>                                                                    | 1,654 distinct raw, <b>1,564 once trimmed and case-folded</b> — 747 rows carry stray whitespace. Those 1,564 become the initial <code>MasterProduct</code> catalogue and its aliases |
| <code>Price</code>                                                                    | <code>unit_price_pvp_eur</code>                                                                                                             | Gross list price before discounts                                                                                                                                                    |
| <code>PromoInd</code>                                                                 | <code>promo_discount_eur</code>                                                                                                             | Per-item promo (17.6 % filled)                                                                                                                                                       |
| <code>PromoGlob</code>                                                                | <code>invoice_allocated_discount_eur</code>                                                                                                 | Only 8 % filled — <b>recompute deterministically</b> from the receipt-level ratio for every row                                                                                      |

| Legacy column       | Model target                                       | Notes                                                                                                                                                                                                                                                                                        |
|---------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Price_Final         | paid_price_eur                                     | Primary net-spend metric; verify against the recomputed value and report drift                                                                                                                                                                                                               |
| Categoria / _2 / _3 | MasterProduct.category_l1_id / l2 / l3             | Fill is <b>100 % / 81.6 % / 52.9 %</b> — far better than an earlier draft claimed. But it is per-row and <b>not trustworthy</b> : one fixture row files a cider under <b>Talho &gt; Vaca &gt; Almondegas</b> . Import as an <b>AUTO</b> suggestion on the product, never as <b>VALIDATED</b> |
| Peso                | weight_observed_kg                                 | Real scale weight (69.7 % filled)                                                                                                                                                                                                                                                            |
| Peso (proposta)     | weight_listed_kg ← MasterProduct.pack_variants     | The legacy <b>VLOOKUP</b> -by-description is exactly the curated-weight learning                                                                                                                                                                                                             |
| Preco/Kg/real       | derived price_per_kg_pvp_eur                       | Legacy name is a misnomer: it is <b>Price / Peso</b> , i.e. the <b>list</b> price per kg                                                                                                                                                                                                     |
| Preco/Kg/promo      | derived price_per_kg_promo_eur                     | <b>(Price – PromoInd) / Peso</b>                                                                                                                                                                                                                                                             |
| Flags (F/f)         | is_fs = true                                       | 425 rows. Never on an invoice — they import as appended Fs articles with the sheet's price as <b>unit_price_pvp_eur</b> , <b>notional_value_source = MANUAL</b> and <b>paid_price_eur = 0.00</b>                                                                                             |
| Preço Fatura        | Receipt.total_eur, taken from the non-Fs rows only | A <b>SUMIF</b> over <b>Full_Date</b> , so on a nudged Fs row it sums only that row — self-referential and meaningless (59 rows equal their own <b>Price_Final</b> ). Never use it as a grouping key                                                                                          |
| Notas / Status      | ReceiptItem.notes                                  | <b>Both hold sparse free text</b> (2.1 % and 2.4 %), not a status enum — <b>Status</b> contains entries like <i>"Devolvido"</i> and <i>"Compras pais dani"</i> . Import both into <b>notes</b> ; <b>Receipt.status</b> is ours, not theirs                                                   |

| Legacy column | Model target               | Notes                                                                                              |
|---------------|----------------------------|----------------------------------------------------------------------------------------------------|
| ID            | ReceiptItem.legacy_row_ref | <b>Not</b> the primary key — PKs are UUID v7. Kept solely for traceability back to the spreadsheet |

**Caveats carried over:** L2/L3 categories are partly blank and partly wrong — import them as suggestions, never as confirmed; **Peso** and €/kg exist for about two thirds of rows; **PromoGlob** was computed on 8 % of rows, so the import must recompute it consistently. The import is idempotent: re-running it updates rather than duplicates.

## Integration Contract

- **Exposes:** **Receipt**, **ReceiptItem**, **MasterProduct**, **ProductPriceHistory**; category spend, loyalty totals, Fs analytics and price trends to Dashboards (M8), each over both **paid\_price\_eur** and **notional\_value\_eur**.
- **Consumes (all defined in the orchestrator brief §1a, never redefined here):** **Entity** — attribution and RBAC; **Merchant** — receipt issuer and parser-profile key; **Category** — the taxonomy this module administers for the **GROCERY** domain; **Tag**; **Document** — original invoice storage and SHA-256 duplicate detection; **Link** — **RECEIPT\_TRANSACTION** edges; **ReviewTask** — the shared Review Queue this module first populates; **Setting** — confidence thresholds and arithmetic tolerance; **AuditLog**; **ImportBatch** and **ProcessingJob** — every upload and the legacy migration. Plus **Transaction** (M2) for ledger reconciliation, which is optional and degrades cleanly when the ledger does not yet exist.
- **Reconciles with:** Banking (M2) via merchant + date ( $\pm 3$  days, configurable) + amount tolerance, emitting **Link(RECEIPT\_TRANSACTION)** edges carrying confidence and decision reasons.
- **Guarantees:** the module is fully functional with **no subscription and no network** — every stage has a local engine, and a remote one is contacted only when explicitly configured; money is decimal EUR everywhere; **paid\_price\_eur** is always what was actually paid; **ProductPriceHistory** is append-only; a category is never silently orphaned by a tree edit.